

CMPE 223 Spring 2026 Programming Homework 3

This assignment is due by 23:59 on Tuesday 28, April 2026.

You are welcome to ask your HW related questions. You should use one of these options:

1. Question Forum: HW Question-and-Answer (Q&A) Forum on LMS is always available. Use the “Forum” link on the course LMS page.
2. Homework **RECITATION HOURS**: There will be two Q&A RECITATION HOURS on the following days:
 - CMPE223 HW3 OfficeHour1: 21.04.2026 Tuesday 05:30-06:30 PM, Zoom ID: <https://tedu.zoom.us/j/98774515954>
 - CMPE223 HW3 OfficeHour2: 27.04.2026 Monday ..., 05:30-06:30 PM, Zoom ID: <https://tedu.zoom.us/j/91804167063>

You are allowed to complete this assignment **as a group of only 2 people**. If you choose to work with a partner, please **add both of your names** to the Excel file I shared on LMS **until 20 April 23:59**. After this deadline, **no more names can be added**, and you will have to **complete the assignment individually**.

Note: Please make sure that you have read the HW document well before participating. However, no HW related questions will be accepted except from the above options.

PROGRAMMING TASK

In this part, you must implement your own tree data structure by taking inspiration from your textbook and use it to help to solve problem. You are not allowed to use any external library or .jar file. Any solutions without using linked list structure are not evaluated!

Question 1 (25 points):

Consider a scenario where you are required to digitally represent the structure of a family that evolves over time. New members join the family either through **birth** or **marriage**, and these relationships must be represented in a hierarchical and structured manner.

To achieve this, a **binary tree** data structure will be used to model the family.

Two fundamental types of events define how the family grows:

- **Birth (BIRTH):** A new individual is born into the family
- **Marriage (MARRIED):** An individual joins the family through marriage

Individuals added through birth are placed into the tree based on their **age**, ensuring that the tree maintains the properties of a **Binary Search Tree (BST)**. This allows the family hierarchy to be organized in a meaningful and consistent way.

In the case of marriage, the newly added individual is not inserted based on age. Instead, the individual is positioned in the tree in a way that preserves family relationships. Specifically, the spouse is added as a child of the **parent of the married individual**, ensuring that married individuals appear at the same level (i.e., as siblings) in the tree.

Through this approach, both **vertical relationships** (parent–child) and **horizontal relationships** (spouses) are represented within a single tree structure.

The program must process a sequence of commands, construct the corresponding family tree, and output the final structure using a specified traversal method.

Input:

First line: N – number of commands.

Next lines: N commands that add new member to a family.

1. BIRTH COMMAND

- Format : BIRTH name age
- Adds a new individual to the family.
- The first individual added becomes the **root** of the tree.
- Individuals are provided in order from **oldest to youngest**.

Insertion Rule:

- The tree must be maintained as a **Binary Search Tree (BST)** based on age:
 - Smaller age → inserted into the **left subtree**
 - Larger age → inserted into the **right subtree**

2.MARRIED COMMAND

- Format : MARRIED name spouseName age
- name: An existing individual in the tree
- spouseName age: The individual joining the family

Insertion Rule:

1. Locate the node corresponding to name.
2. Identify the **parent** of this node.
3. Insert the new individual as a child of this parent.
4. This ensures that spouses are placed at the same level (i.e., as siblings).

Binary Tree Constraint:

- If the parent node:
 - Has an empty left child → insert there
 - Otherwise, if the right child is empty → insert there
 - If both children are occupied → insertion fails (error case)

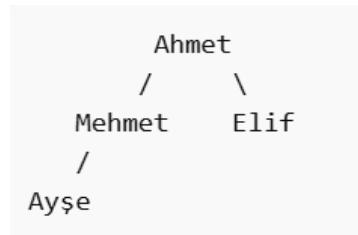
Special Case:

- If the specified individual is the **root** (i.e., has no parent):
 - The spouse should be inserted as the **right child of the root**, if available

Sample Input:

```
4
BIRTH Ahmet 70
BIRTH Mehmet 50
BIRTH Ayşe 30
MARRIED Mehmet Elif 48
```

The tree that will result from the entered inputs is as follows. This is not an actual output; it has been included to provide a clearer explanation.



Output:

After processing all commands, the constructed tree must be printed using **in-order traversal**.

Sample Output:

```
Ayşe Mehmet Ahmet Elif
```

Question 2 (25 points):

TasteChain is a rapidly growing restaurant franchise that manages multiple branches across different regions. The company needs a digital system to represent its organizational hierarchy, from the CEO down to individual staff members at each branch.

To achieve this, you are required to implement a general tree (n-ary tree) data structure to model the restaurant chain's structure. Each node in the tree can have multiple children, representing the hierarchical relationships within the organization.

In this part, you must implement your own tree data structure by taking inspiration from your textbook. You are not allowed to use any external library or .jar file. Any solutions without using linked list structure will not be evaluated!

Each node in the tree should store the following information:

- String name - name of the employee/branch
- int salary - monthly salary

Input:

First line: The filename containing the restaurant chain data.

The program reads the file, constructs the tree, and then prompts the user for operation codes.

File Format:

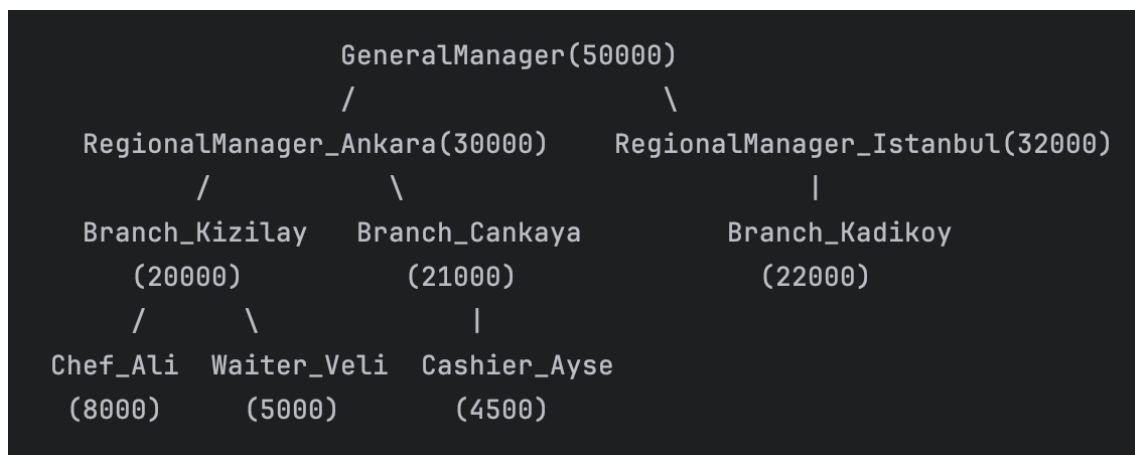
Each line defines a parent-child relationship in the following format:

ParentName ParentSalary, ChildName ChildSalary

- The parent in the first line becomes the root of the tree.
- Names are unique (no duplicate names allowed).
- Salary is an integer value.

Sample File (restaurant.txt):

```
GeneralManager 50000, RegionalManager_Ankara 30000
GeneralManager 50000, RegionalManager_Istanbul 32000
RegionalManager_Ankara 30000, Branch_Kizilay 20000
RegionalManager_Ankara 30000, Branch_Cankaya 21000
Branch_Kizilay 20000, Chef_Ali 8000
Branch_Kizilay 20000, Waiter_Veli 5000
Branch_Cankaya 21000, Cashier_Ayse 4500
RegionalManager_Istanbul 32000, Branch_Kadikoy 22000
```



Your program must read the file, construct the tree using a linked list representation, and provide a menu with the following operations:

Operations:

1. Print Subordinates

- Input: Employee name
- Output: Print all direct children (subordinates) of the given employee
- If employee has no subordinates, print: <name> has no subordinates.
- If the employee is not found, print: Employee not found.

2. Find Common Manager

- Input: Two employee names
- Output: Find and print the Lowest Common Ancestor (LCA) of the two employees
- If either employee is not found, print: Employee not found.

3. Calculate Total Salary

- Input: Employee name
- Output: Calculate and print the total salary of the employee and ALL employees under their subtree (recursive sum)
- If the employee is not found, print: Employee not found.

4. Check Manager

- Input: Two employee names (A, B)
- Output: Check if A is a manager (ancestor) of B
- Print: <A> is a manager of , or <A> is not a manager of .

5. Find Path

- Input: Employee name
- Output: Print the path from root (GeneralManager) to the given employee
- Format: GeneralManager -> RegionalManager_Ankara -> Branch_Kizilay -> Chef_Ali

6. Quit

- Exit the program

Sample Output:

```
Menu:
1. Print Subordinates
2. Find Common Manager
3. Calculate Total Salary
4. Check Manager
5. Find Path
6. Quit

Enter your choice: 1
Enter employee name: RegionalManager_Ankara
Subordinates of RegionalManager_Ankara: Branch_Kizilay, Branch_Cankaya

Enter your choice: 2
Enter first employee name: Chef_Ali
Enter second employee name: Cashier_Ayse
Common Manager: RegionalManager_Ankara

Enter your choice: 3
Enter employee name: Branch_Kizilay
Total salary under Branch_Kizilay: 33000

Enter your choice: 4
Enter first employee name: GeneralManager
Enter second employee name: Waiter_Veli
GeneralManager is a manager of Waiter_Veli.

Enter your choice: 5
Enter employee name: Waiter_Veli
Path: GeneralManager -> RegionalManager_Ankara -> Branch_Kizilay -> Waiter_Veli

Enter your choice: 6
Goodbye!
```

Requirements:

- You MUST use linked list representation for the general tree
- The tree must support any number of children per node
- All operations must work correctly for any valid tree structure
- Handle edge cases (employee not found, single node, etc.)

- **You need to upload your code into VPL on LMS for question.** If you do not upload your code into VPL on LMS, your homework will **not be graded**.
- The Java sources should be WELL DOCUMENTED as comments, as part of your grade will be based on the level of your comments.
- You need to upload **maximum-3 pages** PDF report document that explains your own answers for programming task in a clearly readable PA report format (refer to **PA REPORT FORMAT** section).

PA REPORT FORMAT

A programming assignment report is a self-description of a programming assignment and your solution. The report must not be hand-written. You may use a word processor or the on-line editor of your choice and prepare as a PDF document. The report must be grammatically correct and use complete English sentences. Each report should include the following sections, in the order given:

Information (%2.5): This section includes your ID, name, section, assignment number information properly.

Problem Statement and Code Design (%15): Include a brief summary of the problem and/or your sub-tasks to be completed in this assignment. You should show your modular design rationale by creating a structure chart that indicates your top-down, stepwise refinement of the problem solution. You may create the structure chart using available graphical tools like MS PowerPoint, SmartDraw etc.

Implementation and Functionality (%20): Since you have modular source code, you should describe each sub-module (program) in this section. Each sub-module should include names and types of any input/output parameters as well as the pseudocode algorithm that used for completing its task. By this way, you give meaning to each chart boxes from the previous section.

Testing (%7.5): You should provide a tester class that is able to identify key test points of your program. This class should be able to generate additional (apart from the given sample input/output) test data for the purpose of being clear on what aspects of the solution are being tested with each set. This section should also include a description of any program *bugs* that is, tests which has incorrect results. You should write these to describe your tests, summarize your results, and argue that they cover all types of program behavior.

NOTE: For Question 1, you should test the code with your own family members and include the output in the report.

Final Assessments (%5): In this final section, you should briefly answer the following questions:

1. What were the trouble points in completing this assignment?
2. Which parts were the most challenging for you?
3. What did you like about the assignment? What did you learn from it?
4. Did you use any Artificial Intelligence (AI) tools (e.g., ChatGPT, Copilot, Gemini, etc.) while completing this assignment?
 - a. If yes, specify which tools you used and how you used them (e.g., to clarify a concept, check logic, debug, or generate comments).
 - b. How did you ensure that your final submission reflects your own understanding

and original work?

5. How did AI assistance (if used) affect your learning process?

- a. Did it make problem-solving easier, or did it change how you approached the task?
- b. What are the ethical boundaries you think should apply when using AI in academic work?

GRADING:

- Codes (%50; %25 for Q1, %25 for Q2)
 - Available test cases evaluation on VPL: %15
 - Hidden test cases evaluation: %15
 - Approach to the problem: %20
- Report (%50; %25 for Q1, %25 for Q2)
 - Information: %2.5
 - Problem Statement and Code design: %15
 - Implementation, Functionality: %20
 - Testing: %7.5
 - Final Assessments: %5

IMPORTANT

IMPORTANT NOTES: Do not start your homework before reading these notes!!!

1. **This assignment is due by 23:59 on Tuesday, 28th.**
2. You should upload your homework to LMS before the deadline. No hardcopy submission is needed. You should upload your codes into VPL and your report into submission place on LMS.
3. The standard rules about late homework submissions apply (**20 points will be deducted for each late day**). Please see the course syllabus for further discussion of the late homework policy as well as academic integrity.
4. You ARE NOT ALLOWED to modify the given method names. However, if necessary, you may define additional data members and member functions.
5. Your classes' name MUST BE as shown in the homework description.
6. The submissions that do not obey these rules will not be graded.
7. To increase the efficiency of the grading process as well as the readability of your code, you have to follow the following instructions about the format and general layout of your program.
8. Do not forget to write down your id, name, section, assignment number or any other information relevant to your program in the beginning of your Java files. Example:

```
//-----
// Title: Scheduler tester class
// Author: Name/Surname
// ID: 2100000000
// Section: 1
// Assignment: 1
// Description: This class tests the ...
//-----
```

9. Since your codes will be checked without your observation, you should report everything about your implementation. Add detailed comments to your classes, functions, declarations etc. Make sure that you explain each function in the beginning of your function structure. Example:

```
void setVariable(char varName, int varValue)
//-----
// Summary: Assigns a value to the variable whose
// name is given.
// Precondition: varName is a char and varValue is an
// integer
// Postcondition: The value of the variable is set.
//-----
{
    // Body of the function
}
```

10. Indentation, indentation, indentation...

11. This homework will be graded by your TA, Edanur Özpınar. Thus, you may ask her your homework related questions through [HW forum on LMS course page](#).